
Executive Summary

My program successfully creates a database with the purpose of managing animals and adopters. My program can add and list adopters as well as their preferences; add and list animals and their adoption history. My program allows an adopter to adopt multiple animals simultaneously, and un-adopt them freely; adopters can never readopt an animal. My program is capable of only showing the user animals that match a specified adopter's preferences, it follows my program is also capable of asking for and parsing an adopter's preferences.

My program can create multiple database tables and is fully capable of storing and retrieving data from these tables, as well as successfully allowing tables to access the data of another. My program can also work with a pre-existing database (i.e. records already exist in the database not added by the program).

It successfully separates database interactions, control logic, and a user interface, while successfully utilising an ORM to manipulate and manage data.

My program passes all the supplied tests, as well as successfully passing my own tests.

Code Design

General Design

I used JPA and EclipseLink, while they give me less direct control over the database, I find it more suitable as a program begins to scale, as it can efficiently handle queries and the relationships between database tables with less code (in my experience), features such as collection tables are also very convenient and reduce the need for direct interaction with the database. Additionally, the code presents itself as quite verbose due to the required JPA annotations, which could allow another programmer to understand with greater clarity than monolithic native SQL statements. The natural abstraction provided by DAOs and their relevant entities was also particularly useful in program organisation and decomposition.

My program decomposes control logic, database interactions, and a user interface. Control logic is handled by Practical3 (the main), database interactions are handled by the DAOs and the JPA entities (DAOs handle queries exclusively), and the user interface is handled by the dedicated class 'userInterface'.

Automatic Primary Key Generation

As per the spec, Adopter is required to have an auto-incrementing integer primary key; hence I used JPA's provided key generation methods, I used the 'Identity' generation type¹, this provides exactly the desired behaviour.

Database Relationships and Normalisation

At no point does my database have a many-to-many relationship. With both animals and adopters having a one-to-many relationship with adoptions, and adopters having a one-to-many relationship with 'preferred_species'. This (under my observation) follows the three principles of database normalisation, hence can be assumed to be appropriately well organised and normalised database.

Composite Keys and Collection Tables

¹ [An Overview of Identifiers in Hibernate/JPA | Baeldung](#)

The adoption table and the preferred_species table are both required to have composite primary keys. While I believe there are several ways to this, I chose to have a separate class that represents the composite key, using JPA annotations, I can reference this class and subsequently set 2 columns at the primary key, and JPA converts it to a composite key², I do this only for adoptions.

Preferred species is different; due to the simplicity of the data it stores (each record is only two columns), I used a collection table³, this reduces the number of classes by 2 (preference entity and preference's id class) and condenses all the code related to producing the table into only a few lines, while losing nothing in return. A collection table however can only be used when a table is tied directly to one table in particular, (i.e. Preferred_Species is directly tied to Adopter and nothing else) hence why it cannot be used with the adoption table.

DAOs and Entities

In my program, DAOs are the only part that directly access the database through queries. As expected, entities must exist for the ORM to function, the attributes entities come together to form the required schema.

I use 'TypedQueries' instead of standard queries⁴. TypedQueries return a predetermined object type. This has the same behaviour as regular queries, as regular queries require an explicit type conversion anyway (hence essentially having a predetermined object type). Using a TypedQuery comes with the bonus of removing the need for an 'unchecked' type cast warning suppression, and in my opinion, seems to be much better practice.

Date Formatting

When using the program, all dates appear in the standard date format (YYYY-MM-DD). When data is viewed using SQLite3 directly, it is observed it as a Unix timestamp. Prof. Ruth Letham stated in a supplied FAQ document that this is acceptable behaviour, hence I decided to leave it unchanged, as it passes the supplied tests, and is formatted correctly when accessed directly via the program.

Input validation

Inputs are not specifically validated based on what information they hold. For example, a phone number can be 1 digit long. I do not enforce extra constraints as it was not specified in the specification. Adding extra constraints would likely add extra, unwanted functionality, and would change the UI sequence.

Testing

My testing mainly focused on input validation, as well as making sure there were no contradictions in the database logic, especially the queries within the DAO. These are the most likely to throw errors as they are the most fragile, and the likeliest error to slip through stacscheck. It also covers some basic SQL injection, to ensure the program still functions correctly under malicious inputs.

It may not seem necessary to test my program's ability to handle SQL injection attempts⁵, in terms of security. However, if an SQL injection attempt is successful, that means my program will provide an unexpected output. I do not want any unintentional behaviour in my program, so I test against it.

² [Composite Primary Keys in JPA | Baeldung](#)

³ [EclipseLink/Examples/JPA/2.0/ElementCollections - Eclipsepedia](#)

⁴ [Types of JPA Queries | Baeldung](#)

⁵ [SQL Injection](#)

For the UI, we were given a working UI class, while I expanded upon it by adding new methods, all the original methods can be assumed to be trivially correct. However, methods that I added to the UI (ensuring a given ID corresponds to an actual valid adopter/animal) are tested in-depth.

Additionally, the input system provided means that it is likely impossible to add a null value to the database when not intended, as all input prompts loop indefinitely until the user enters a valid input. This means the database is seldom at risk of improper records such as null primary key exceptions etc.

So, with all these considerations in mind, my testing mainly covers input validation and database logic as these are the most error-prone areas. Database logic can be tested by creating sample databases and comparing the expected output to the observed output for discrepancies, as well as directly testing known flags (e.g. has an animal been adopted by this person before?). Input validation is mostly making sure an input isn't blank. For an ID input, it must be compared against a list of valid ids, again, I can use a small sample database to ensure allowed id was valid. This should ensure my program's formatting and logic generally holds, at which point I can assume it to be robust under most circumstances a user may experience.

Lessons Learned

Naturally, this practical taught me a lot about databases and how to use them effectively. However, JPA proved to be exceptionally hard to debug, with very sparse and often ineffective resources available, and given the black box like nature of the error messages, this program proved especially difficult to debug, not aided by the subtle differences between JPQL and SQL. Additionally, JPA requires more set-up, planning and organisation than JDBC. However, I understand that for what JPA requires in time, it makes up for in robustness, as the usage of entities and DAOs naturally lends JPA to clean encapsulation. I also much prefer the object-oriented way of interacting with the database, and I believed it proved its usefulness as I progressed in the project. Hence, were I to work with databases in the future, I believe using an ORM would be my preferred approach, though I would attempt to find a more accessible alternative to JPA.

Appendix

My program passes 30/30 of the supplied tests.

Testing part 1

Test	Arguments Input	Input	Expected output	Pass/Fail
Adding an animal	testing/testBatch1.db 1	D001 smaug dragon drake male greedy 2026-04-06	Stored smaug D001 male dragon (drake) arrived 2026-04-06 greedy	PASS
Adding an animal with the same ID	testing/testBatch1.db 1	D001 smaug dragon drake male greedy 2026-04-06	Could not add animal.	PASS
Attempting to add an animal with a	testing/testBatch2.db 1		Animal name cannot be blank.	PASS

blank name				
Attempting to add an animal with a blank species	testing/testBatch2.db 1	D001 smaug	Animal species cannot be blank	PASS
Adding an animal with an improper date.	testing/testBatch2.db 1	D001 smaug dragon drake male greedy 1233123223	Invalid date. Use YYYY-MM-DD.	PASS

Testing part 2

Test	Arguments Input	Input	Expected output	Pass/Fail
Adding an adopter	Testing/testBatch3.db 2	sauron 12345 darklord@lotr.com Barad-dur dragon	Stored 1 sauron 12345 darklord@lotr.com barad-dur	PASS
Adding a duplicate adopter	testing/testBatch3.db 2	sauron 12345 darklord@lotr.com Barad-dur dragon	Stored 2 sauron 12345 darklord@lotr.com barad-dur	PASS

Testing part 3

Test	Arguments Input	Expected output	Pass/Fail
Listing adopters where one adopter exists	Testing/testBatch4.db 3	1 sauron 12345 darklord@lotr.com barad-dur Preferred species: dragon	PASS
Listing adopter when no adopters exist	testing/empty.db 3	[no output]	PASS

Testing part 4

Test	Arguments Input	Expected output	Pass/Fail
Listing animals [which have no adoptions]	testing/testBatch5.db 4	smaug D001 male dragon (drake) arrived 2026-04-06 greedy	PASS
Listing animals [which are currently adopted]	testing/testBatch5.db 4	smaug D001 male dragon (drake) arrived 2026-04-06 greedy Adopted 2026-04-01 - present by sauron	PASS
Listing animals [which are currently adopted]	testing/testBatch5.db 4	[...] Adopted 2026-04-01 - 2026-05-06 by sauron Adopted 2026-01-01 - present by frodo	PASS

with a previous adoption]			
Listing animals [which multiple previous adoptions]	testing/singleEntry.db 4	[...] Adopted 2026-04-01 - 2026-05-06 by sauron Adopted 2026-01-01 - 2026-12-12 by frodo	PASS
Listing all animals where no animals exist	testing/empty.db 4	[No output]	PASS

Testing part 5

Test	Arguments Input	Input	Expected output	Pass/Fail
Listing matches where no animals are present in the db [and one adopter was added]	testing/testBatch6.db 5	1	1 frodo baggins 123 frodo@shire.com Bagend Preferred species: dragon	PASS
Listing a valid match	testing/testBatch7.db 5	1	1 frodo baggins 123 frodo@shire.com Bagend Preferred species: dragon Smaug D001 male dragon (drake) arrived 2026-04-06 greedy	PASS
Trying to list available animals when an adopter has a previous, terminated adoption record	testing/empty.db 5	1	1 frodo baggins 123 frodo@shire.com Bagend Preferred species: dragon	PASS
Entering an invalid adopter ID	testing/empty.db 5	-1	Invalid adopter id.	PASS

Testing part 6

Test	Arguments Input	Input	Expected output	Pass/Fail
Adopting animal that matches the user's species preference.	Testing/testBatch8.db 6	1 D001	dragonborn adopted alduin on 2026-04-07	PASS
Adopting an animal that does not match the user's species preference.	Testing/testBatch8.db 6	1 V1	dragonborn adopted V1 on 2026-11-16 [where the species of V1 is not a preference of 'dragonborn']	PASS
Adopting an animal that an adopter is currently adopting	Testing/sample.db 6	2 P001	Invalid animal id.	PASS
Adopting an animal that has previously been adopted and unadopted by the same adopter.	Testing/sample.db 6	2 S005	Invalid animal id.	PASS

Adopting an animal using an invalid adopter ID	Testing/sample.db 6	444	Invalid adopter id.	PASS
Adopting an animal using an invalid animal ID	Testing/sample.db 6	1 INVALID_ID	Invalid animal id.	PASS

Testing part 7

Test	Arguments Input	Test	Expected output	Pass/Fail
Unadopting an animal that exists and has an active adoption	Testing/testBatch9.db 7	V1	V1 returned to centre on 2026-11-23	PASS
Unadopting an animal that does not exist	Testing/sample.db 7	INVALID_ID	Invalid animal id.	PASS
Unadopting an animal that doesn't have an active adoption	Testing/sample.db 7	S008	Invalid animal id.	PASS

Sequence Tests: Testing a specific sequence of steps, documenting the results of each step.

Sequence Test 1: Adopting a new animal with a new adopter

Test	Arguments Input	Input	Expected output	Pass/Fail
Creating a new animal	Testing/empty.db 1	D001 alduin dragon ancient male evil 2026-09-04	Stored Alduin D001 male dragon (ancient) arrived 2026-09-04 evil	PASS
Creating a new adopter	Testing/empty.db 2	Dragonborn 13579 Dragonborn@skyrim.com whiterun	Stored 1 Dragonborn 13579 dragonborn@skyrim.com whiterun	PASS
Adopting the animal	Testing/empty.db 6	1 D001	Dragonborn adopted Alduin on 2026-09-04	PASS

SequenceTest 1.1: (Continuing from SequenceTest 1) Unadopting animal then attempting to readopt them

Test	Arguments Input	Input	Expected output	Pass/Fail
Unadopting animal	Testing/empty.db 7	D001	Alduin returned to centre on 2026-09-06	PASS
Adopt animal	Testing/empty.db 6	D001	Invalid animal id.	PASS
List adoptions [to check animal adoption records]	Testing/empty.db		Alduin D001 male dragon (ancient) arrived 2026-09-04 evil	PASS

			Adopted 2026-09-04 - 2026-09-06 by Dragonborn	
--	--	--	---	--

SQL Injection

Test	Arguments Input	Input	Expected output	Pass/Fail
Adding an animal of id of "D001 OR 1 = 1" and then getting preferences	Testing/sample.db 5	D001 OR 1=1 SQL Injection [...]	1 Jane Doe 412345678 jane.doe@example.com 123 Maple St, Sydney [...] Peaches P004 male pig (Hampshire) arrived 2023-05-03	PASS
Adding an animal with species "spider OR 1=1" and then getting preferences	Testing/sample.db 5	V1 v1 Spider OR 1=1 Robot aa 2026-06-06	1 Jane Doe 412345678 jane.doe@example.com 123 Maple St, Sydney [...] Peaches P004 male pig (Hampshire) arrived 2023-05-03	PASS
Adding an animal with id [" or ""="]	Testing/sample.db 5	V1 " or ""=" " V1 V1 V1 V1 2026-01-01	1 Jane Doe 412345678 jane.doe@example.com 123 Maple St, Sydney [...] Peaches P004 male pig (Hampshire) arrived 2023-05-03	PASS

References

1. PK generation strategies [An Overview of Identifiers in Hibernate/JPA | Baeldung](#)
2. Composite keys [Composite Primary Keys in JPA | Baeldung](#)
3. Collection tables and Element collections [EclipseLink/Examples/JPA/2.0/ElementCollections - Eclipsepedia](#)
4. TypedQuery [Types of JPA Queries | Baeldung](#)
5. Basic SQL injection techniques [SQL Injection](#)